



RAG, from first principles

A reference for everything we built in the workshop

SkillCred · AI/ML Stream

Keep this. It answers the questions you'll be asked in an interview.

What this document is

In the session we built a working retrieval engine from an empty file. This document explains **why** each piece works, in more depth than we had time for live.

It is written to be read once end to end, then used as a reference. Every section ends with the questions an interviewer is likely to ask about that concept.

Contents

1. **The problem** — why a language model alone is not enough
2. **Chunking** — cutting documents into searchable pieces
3. **Embeddings** — turning meaning into numbers
4. **Similarity** — measuring how close two meanings are
5. **Retrieval** — finding and ranking the best passage
6. **Grounding** — the actual RAG moment
7. **LLM-as-judge** — using a model to score an answer
8. **Structured output** — making the result usable by code
9. **Adaptive retrieval** — changing what you search, not how you ask
10. **Where quality actually comes from**
11. **Glossary**

1. The problem

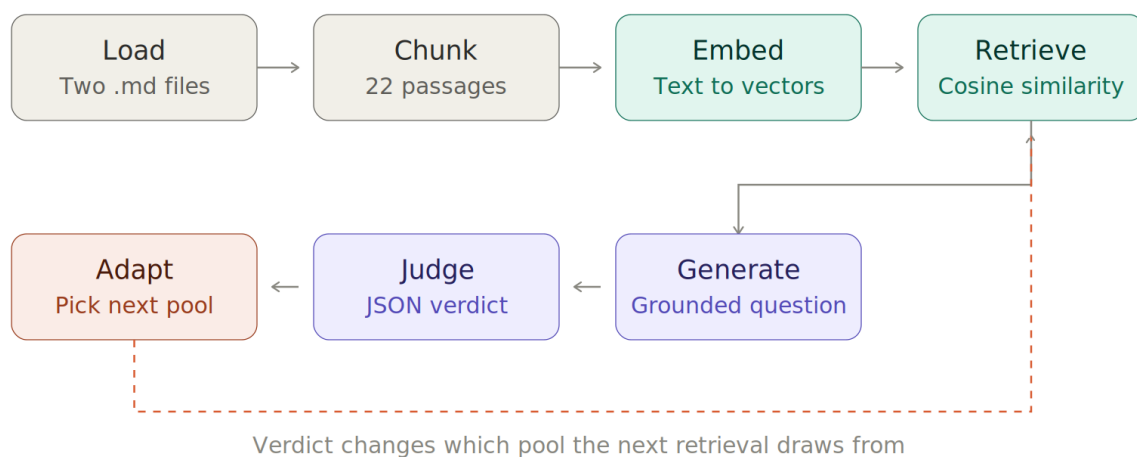
Ask a language model a question and it answers from a compressed statistical impression of everything it read during training. There is no page it can point to. That produces three failures that matter for anything serious:

FAILURE	WHY IT HAPPENS
It invents things	The model predicts likely text. Plausible and true are different properties.
No source	There is no stored document to cite. The knowledge is distributed across weights.
You cannot correct it	If it is wrong, there is nothing to edit short of retraining.

RAG — Retrieval Augmented Generation — solves all three by supplying the document at question time.

- **Retrieval** — find the relevant passage in a collection you control
- **Augmented** — insert that passage into the prompt
- **Generation** — let the model write using it

Closed-book exam versus open-book exam. Same student, very different answers.



THE KEY CONSEQUENCE

Behaviour becomes editable. If the bot asks a bad question, you fix the note, not the model. That is the whole reason RAG dominates production systems.

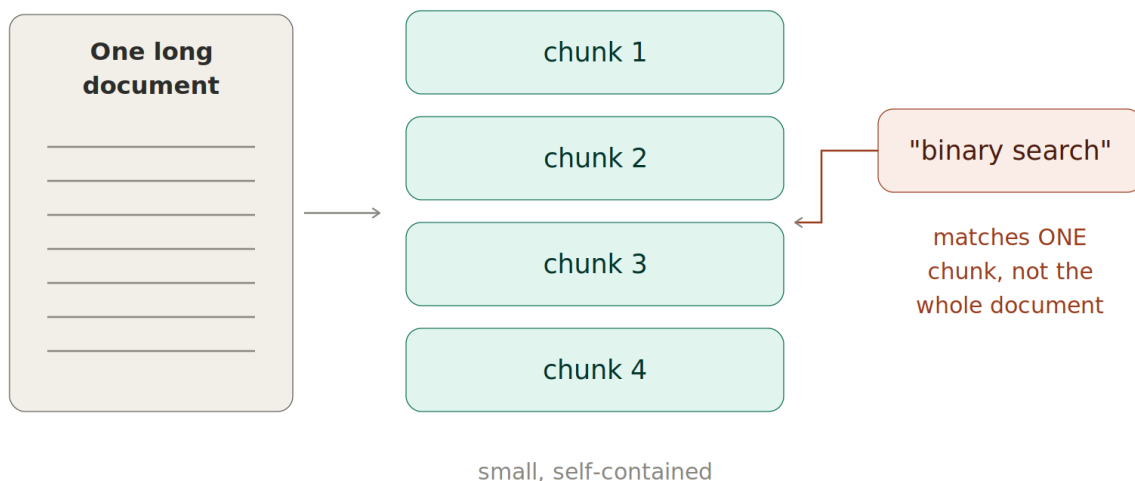
Interview questions

- When would you use RAG instead of fine-tuning?
- What can RAG **not** fix?

Short answers: RAG when the knowledge changes often, must be cited, or is private. Fine-tuning when you need a different **behaviour** or style rather than different facts. RAG cannot fix a model that reasons badly — it only controls what the model is reasoning about.

2. Chunking

You cannot search a whole document usefully. If every search returns fifty pages, you have moved the problem rather than solved it. So documents get cut into **chunks** — pieces small enough that one piece is one answer.



Cut too big and search returns noise. Cut too small and meaning is lost.

Two strategies

Fixed-size chunking

Cut every N characters. Simple, works on anything, and completely blind to meaning. It will cut a sentence in half without hesitation.

Semantic chunking

Cut where the document already divides — headings, sections, paragraphs. The author has already told you where the ideas separate; using that is free information.

In the workshop we used both: semantic first, then fixed-size as a fallback for anything still too long. That is the standard production pattern.

```
parts = section.split("### Deeper layer")    # 1. semantic
for part in parts:
    for j in range(0, len(part), max_chars): # 2. size fallback
        ...
```

THE FAILURE YOU SAW LIVE

Our fixed-size fallback produced a chunk containing four characters: `ns?"` — the tail of a cut sentence. Meaningless, and sitting in the search index waiting to be retrieved. The fix was a `min_chars` floor. Every RAG system needs one.

The trade-off, stated properly

CHUNKS TOO BIG	CHUNKS TOO SMALL
Search returns mostly irrelevant text	Meaning is cut in half
The model gets buried in noise	"It is O(n) because..." — what is?
You pay for tokens you don't need	The passage no longer stands alone

There is no universally correct size. 200–800 characters is a common starting range for prose; code and tables usually want chunking by structure instead.

Overlap

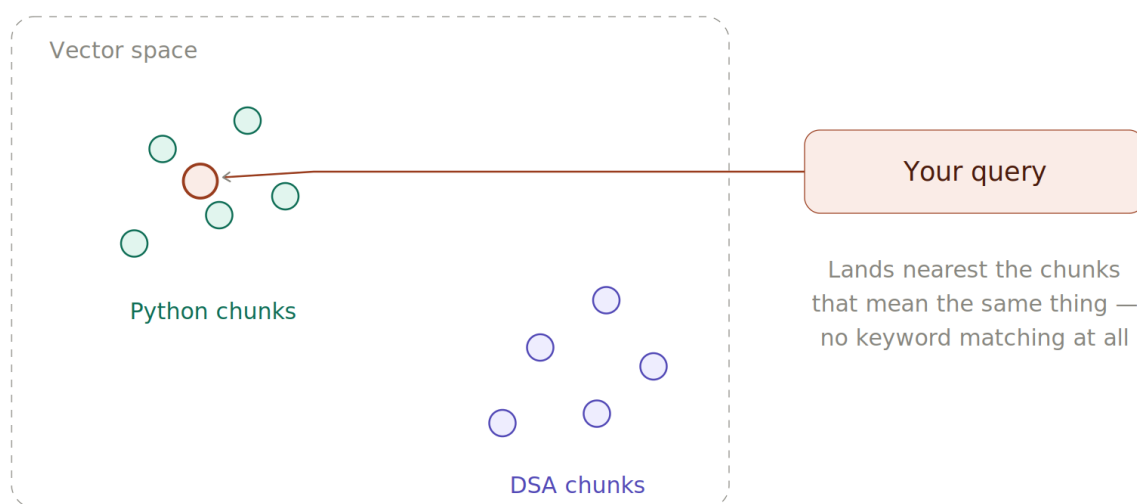
A common refinement is to let chunks overlap by 10–20% so an idea spanning a boundary appears whole in at least one chunk. It costs storage and duplicate results; it buys robustness.

Interview questions

- How would you chunk a large document, and what happens if chunks are too big or too small?
- How would you chunk source code differently from prose?
- What is chunk overlap and when is it worth the cost?

3. Embeddings

Computers cannot compare meaning. They can compare numbers. An **embedding model** reads a piece of text and produces a fixed-length list of numbers — a **vector** — arranged so that texts with similar meaning produce similar vectors.



Our model produced 384 numbers per chunk. Larger models produce 768 or 1536. Nobody reads these numbers; the only thing that matters is how close two vectors are.

Why this beats keyword search

Consider the question **"how do python dictionaries work"** and the passage **"a hash table stores key-value pairs for fast lookup"**. They share **zero** words. A keyword search scores this pair at nothing.

Keyword search compares **letters**. Embedding search compares **meaning**. Technical writing is full of synonyms — dictionary/hash table/map, fast/O(1)/constant time — and full of words with several meanings, so letter matching fails in both directions.

KEYWORD SEARCH	EMBEDDING SEARCH
Compares letters	Compares meaning
Fast, cheap, no model needed	Needs a model to produce the vectors
Excellent for exact terms, names, IDs	Excellent for questions in plain English
Blind to synonyms	Can miss a rare exact term

HYBRID SEARCH

Neither is universally better, so production systems frequently run both and combine the scores. That combination is called **hybrid search** and it is worth knowing the name — it comes up in interviews constantly.

Where the model runs

We ran the embedding model **locally** — 67 MB, on the laptop, no API and no cost — while the question-writing model ran in the cloud. That split is a real architectural decision:

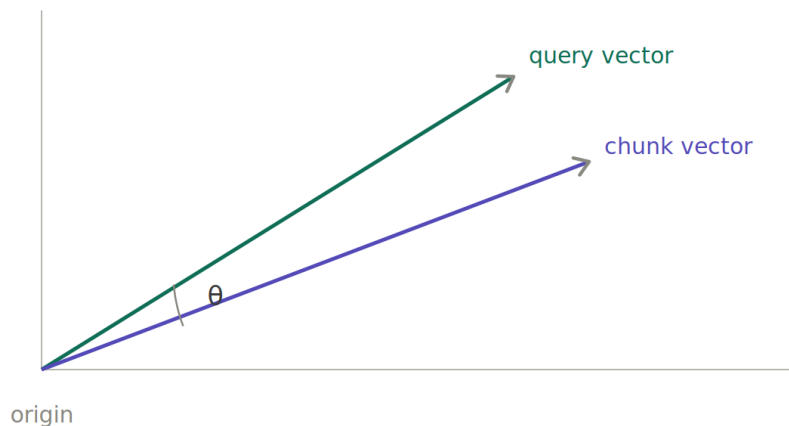
- **Local:** no per-call cost, no rate limits, documents never leave the machine, works offline. Costs you a download and some RAM.
- **API:** nothing to install, best-in-class models, scales without your hardware. Costs money per call, and your documents travel.

Interview questions

- Why would you run embeddings locally rather than calling an API?
- Your retrieval is poor. How would you tell whether the embedding model is the problem?
- What happens if your documents are in a language the model wasn't trained on?

4. Similarity

Two vectors of 384 numbers. How close are they?



small θ = similar meaning = high score

$\text{np.dot}(a, b) / (\text{norm}(a) * \text{norm}(b))$ measures the angle, not the length

Picture each vector as an arrow from the origin. **Cosine similarity** measures the **angle** between two arrows, ignoring their length.

```
def similarity(a, b):  
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

The result runs from 1.0 (identical direction) down to 0 (unrelated) and, in principle, to -1 (opposite). In practice most embedding models produce values in a narrow band — which is why you should compare scores **within** a model and never **across** models.

A TRAP WORTH KNOWING

Model A scoring 0.87 and model B scoring 0.62 does not mean A is more confident. Different models use different portions of the scale. To compare fairly, normalise against each model's own spread.

Why the angle and not the distance?

Length tends to track how much text there is. A long passage and a short one about the same topic should still match, so we discard length and keep direction.

Interview questions

- Why cosine similarity rather than Euclidean distance?
- What does a similarity of 0.6 mean? (Trick: nothing, without context.)

- How would you decide a threshold below which you refuse to answer?

5. Retrieval and ranking

Scoring gives one number per chunk. With 24 chunks you have 24 numbers — but you can only hand the model one passage. So you rank.

```
scored = [
```

0.87	python_dictionary_lookups_base
0.81	dsa_hash_tables_base
0.62	python_generators_base

```
]
```

the score

the chunk dict

```
for _, c in scored[:top_k]
```

discard the score, keep the dict — that is what the underscore means

```
scored = [(similarity(q, c["vec"]), c) for c in chunks] # (score, chunk) pairs
scored.sort(key=lambda x: x[0], reverse=True)          # best first
return [c for _, c in scored[:top_k]]                  # keep the chunk
```

The **tuple** is the important part. Sorting a list of scores alone would tell you the best score but not which passage earned it. Pairing them means both move together.

top_k — how many passages to pass on

- **top_k = 1** — precise, cheap, and fails when the answer spans two passages
- **top_k = 3-5** — more robust, more tokens, more chance of including noise

More context is not automatically better. Irrelevant passages actively degrade the output.

Scaling beyond a loop

Our search compares against every chunk — fine for 24, unworkable for 100,000. Vector databases (Chroma, FAISS, pgvector, Pinecone) use approximate nearest-neighbour indexes to find close vectors without checking all of them, trading a little accuracy for a large speed gain.

Re-ranking

A common two-stage pattern: retrieve the top 20 cheaply, then use a slower, more accurate model to reorder those 20 and keep the best 3. Fast filter, accurate finish.

Interview questions

- What breaks when you go from 20 chunks to 10 million?
- What is re-ranking and why bother?
- How would you evaluate whether your retriever is any good?

6. Grounding

This is the actual RAG moment, and it is simpler than people expect: **paste the retrieved passage into the prompt.**

```
prompt = f"""You are a technical interviewer.
Using ONLY the reference material below, ask ONE interview question.

Reference material:
{context['text']}
"""
```

The word **ONLY** is doing real work. Without it the model treats the passage as a suggestion and blends it with its own recollection.

HOW TO CHECK IT WORKED

Read the output, then read the passage. Every specific claim in the output should appear in the passage. Anything else is the model filling gaps — which means your instruction was not tight enough, or your retrieval returned the wrong passage.

What grounding buys you

WITHOUT GROUNDING	WITH GROUNDING
Answers from vague memory	Answers from a specific passage
Cannot cite a source	You know exactly which one
If wrong, nothing to fix	Edit the note, change the behaviour

Interview questions

- How do you stop a grounded model from adding facts of its own?
- What do you do when retrieval returns nothing relevant?

On that last one: the correct behaviour is to say so. A system that answers confidently from an irrelevant passage is worse than one that admits it found nothing.

7. LLM-as-judge

We used a second, separate model call to grade an answer against the same passage.

Two design points matter:

- **Separate call.** If one call both asked and graded, the model would be marking its own homework, and it is generous to itself.
- **Graded against the passage**, not against the model's opinion. The rubric is the retrieved text.

Known weaknesses

- **Agreeableness.** Models tend to award decent scores to confident-sounding answers. Counter it by demanding specific terms before high marks.
- **Position and verbosity bias.** Longer answers score higher than they should.
- **Inconsistency.** The same answer graded twice can differ. Use `temperature=0`, and for anything serious, grade more than once.

DO NOT USE IT UNCHECKED FOR ANYTHING THAT MATTERS

LLM-as-judge is excellent for practice, feedback and rough scoring. For a decision that affects someone — a hiring outcome, a certificate — it belongs alongside a human, not instead of one.

Interview questions

- How do you know your judge isn't just agreeing with the candidate?
- How would you measure whether your judge agrees with a human grader?

8. Structured output

The grader's verdict has to be usable by code. An `if` statement can read `"weak"`; it cannot read a paragraph of English. So we demand JSON:

```
{"correctness": 3, "clarity": 4, "feedback": "...", "verdict": "weak"}
```

And it will fail

A language model predicts likely text. It has no mechanism guaranteeing valid JSON — your instruction is a strong request, not a rule. It may:

- open with "Sure! Here's the grading:"
- wrap the JSON in markdown code fences
- add an explanation after the closing brace

Perhaps one call in fifty. Rare enough to survive your testing, common enough to break in front of an audience.

```
try:
    return json.loads(raw)
except json.JSONDecodeError:
    start, end = raw.find("{"), raw.rfind("}")
    return json.loads(raw[start:end + 1])    # cut out the friendly words
```

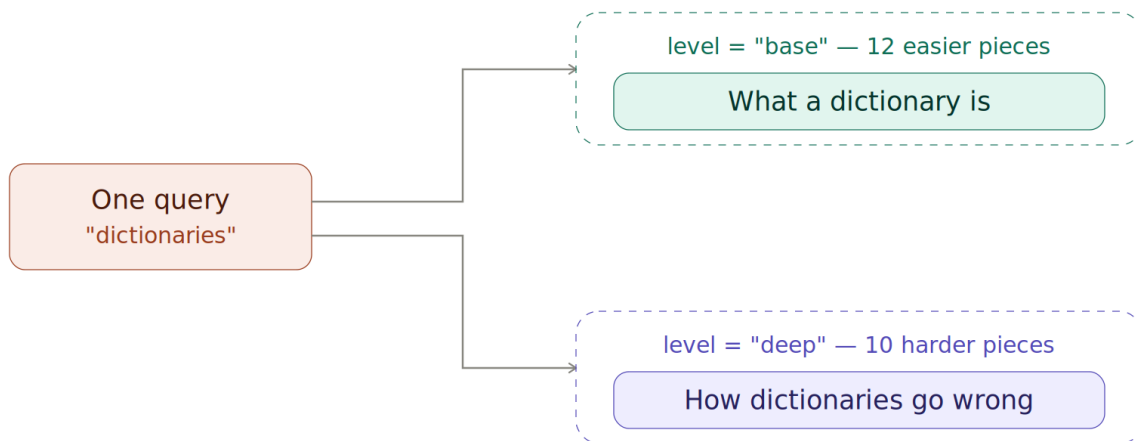
THE GENERAL LESSON

Anything a model returns is a suggestion, not a guarantee. If your next line of code depends on its shape, you must verify that shape and have a plan for the failure. Newer APIs offer a strict JSON mode — use it, and still handle the failure.

9. Adaptive retrieval

Our bot gets harder when you answer well. The entire mechanism is one line:

```
level = "deep" if verdict == "strong" else "base"
```



Same query, same prompt. Only the filter changed.

Every passage carries a `level` label. The verdict decides which pool the next search draws from. Notice what did **not** happen: we did not make the prompt cleverer, and we did not change models.

Metadata filtering, generally

The `level` label is **metadata** — information about a chunk that isn't its text. Filtering on metadata before searching is one of the most useful techniques in RAG:

- Restrict to a date range, so old documents cannot be retrieved
- Restrict by department, so users only see what they may see
- Restrict by document type, language, or author
- Restrict by difficulty — which is exactly what we did

Access control in a RAG system is usually metadata filtering. Get it wrong and you leak documents.

10. Where quality actually comes from

In the session we removed the `level` filter and the adaptive behaviour vanished. No prompt changed. No model changed. We changed **which material was searchable**.

THE SINGLE MOST USEFUL SENTENCE IN THIS DOCUMENT

Most of your quality comes from retrieval, not from wording. When a RAG system disappoints, the instinct is to rewrite the prompt. Usually the real problem is what got retrieved — bad chunks, a missing filter, or the wrong passage winning.

A debugging order that works

1. **Look at what was retrieved.** Print it. Was it the right passage? If not, stop — the prompt is not your problem.
2. **If retrieval is wrong:** check chunking first, then the embedding model, then your filters.
3. **If retrieval is right but the output is poor:** now it is the prompt.
4. **If the output is right but inconsistent:** lower the temperature, or the task is genuinely ambiguous.

Most people start at step 3. Starting at step 1 will save you more time than any other habit in this document.

How to know a change helped

Write twenty questions where you already know which passage should win. Measure how often retrieval returns it. Change one thing. Measure again.

Without this you are guessing, and guessing feels like progress for a long time.

11. Glossary

TERM	MEANING
RAG	Retrieval Augmented Generation. Find a relevant document, put it in the prompt, let the model write from it.
Chunk	One searchable piece of a document.
Embedding	A list of numbers representing the meaning of a text.
Vector	The same thing, said in maths.
Cosine similarity	The angle between two vectors. Higher means more similar.
Retrieval	Finding the most relevant chunks for a query.
top_k	How many chunks to keep after ranking.
Grounding	Putting the retrieved text into the prompt so the model answers from it.
Context window	The maximum text a model can read at once.
Hallucination	A confident, fluent, false statement.
LLM-as-judge	Using a model to score output, usually against a reference.
Structured output	Forcing a machine-readable reply, usually JSON.
Metadata filtering	Narrowing the search using labels on chunks, before comparing vectors.
Hybrid search	Combining keyword and embedding scores.
Re-ranking	Retrieve many cheaply, then reorder the shortlist accurately.
Vector database	Storage built for fast approximate nearest-neighbour search.
Chunk overlap	Letting chunks share text so ideas at a boundary survive.
Temperature	How much randomness in the model's output. 0 is the most repeatable.

The seven questions

If you can answer these, you understand RAG better than most people who list it on a CV.

1. Why is grading a **separate** model call from asking the question?
2. What breaks when you go from 24 chunks to 10,000?
3. How would you tell whether **retrieval** or **generation** is your weak point?
4. Why force JSON instead of parsing English?
5. Why does a metadata filter beat asking the model for "a harder question"?
6. How would you stop the grader from simply agreeing with the candidate?
7. What happens if your chunks are too big? Too small?

NEXT

The take-home notebook has the second half of the system — grading, adapting, the full interview loop and a web app. The assignment sheet has 25 things to build with it. Pick one and finish it.